2026 Spring

Course ID: 1142-1765

AdvancedProgramming

04-Python programming for ˙Valid Parentheses˚

Advanced Programming

Python · C++ · Rust

Data Structures & Algorithms

Python C++ Rust

Dynamic Programming · Divide & Conquer · Greedy Algorithms · Arrays · Lists · Maps & Sets

Part-1

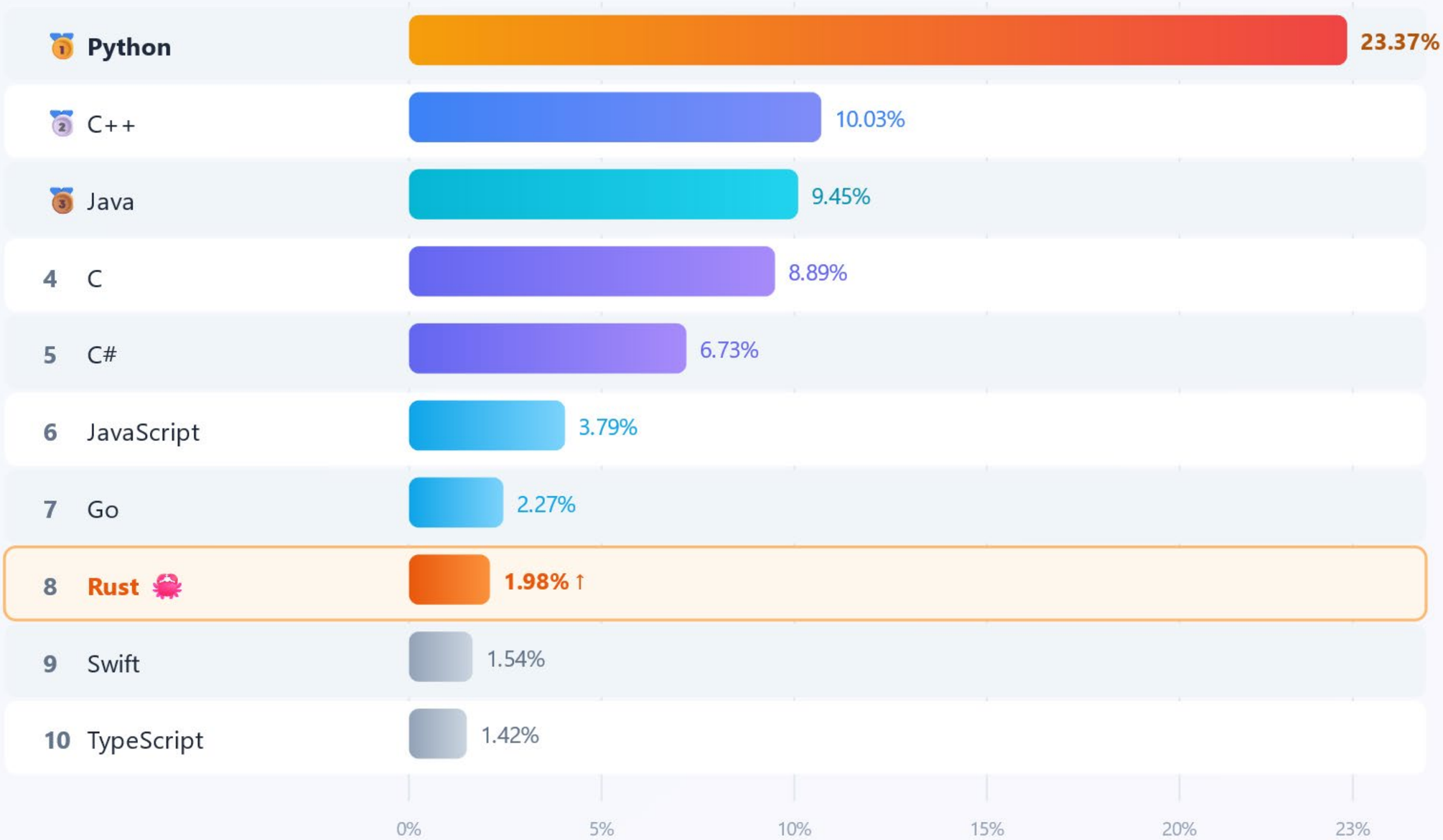
Python Data Structures

LeetCode Problem "Valid Parentheses"

Course Activities
Github repository
Discussion Forum

🏆 Top 10 Programming Languages

Source: TIOBE Index · November 2025



🦀 Rust reached its all-time high of #8 in Nov 2025 — memory safety & performance driving adoption

Python

Data Structures 2

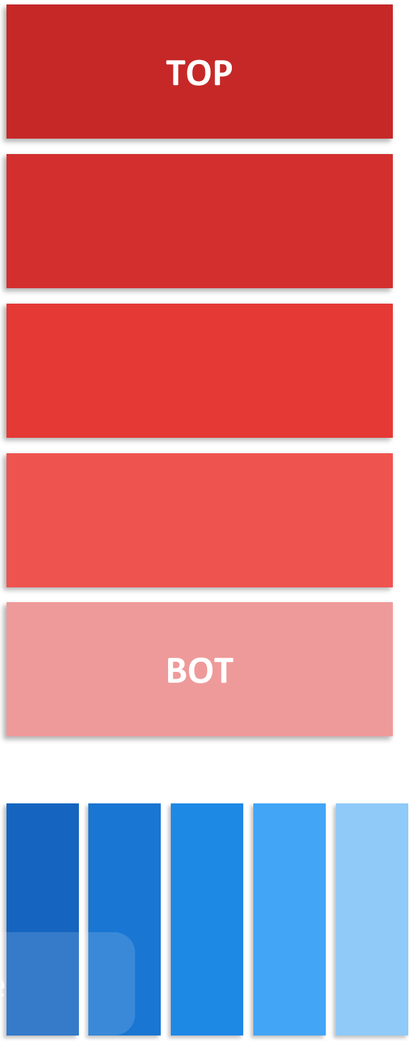
Stack and Queue

Comprehensive Guide to Data Structures Stack and Queue

Stack & Queue

in Python

Fundamental Data Structures for Every Programmer



Agenda

01 **Stack Concept**

LIFO principle & real-world analogy

02 **Stack in Python**

list, deque, LifoQueue implementations

03 **Stack Applications**

Expression parsing, undo/redo, DFS

04 **Queue Concept**

FIFO principle & real-world analogy

05 **Queue in Python**

list, deque, Queue implementations

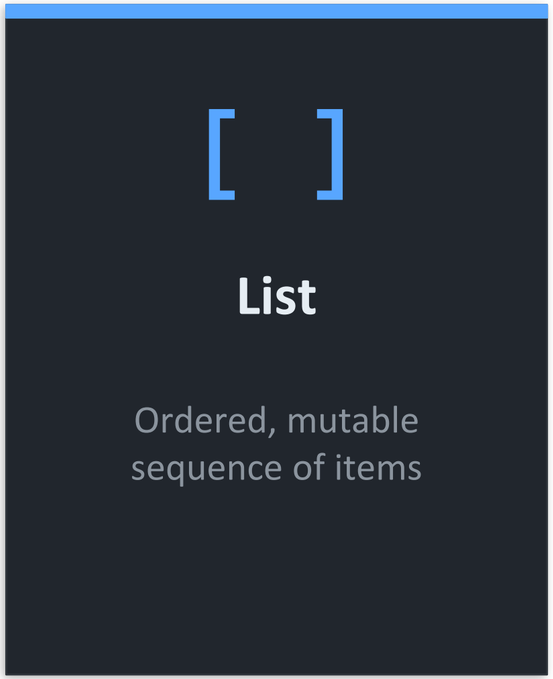
06 **Queue Applications**

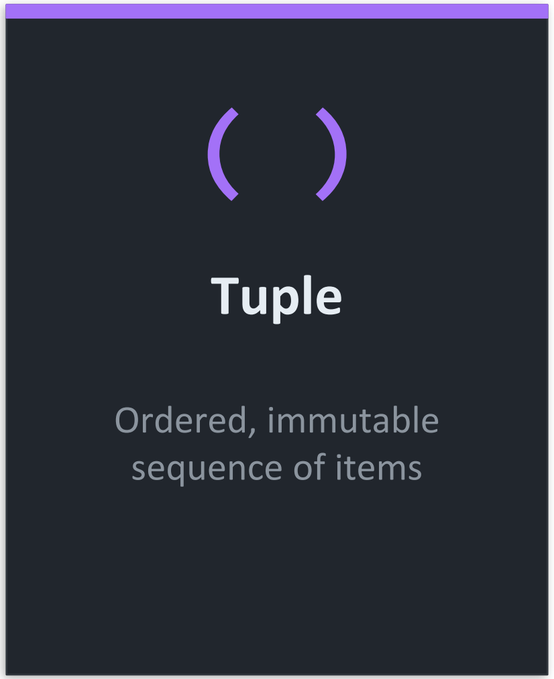
BFS, task scheduling, buffers

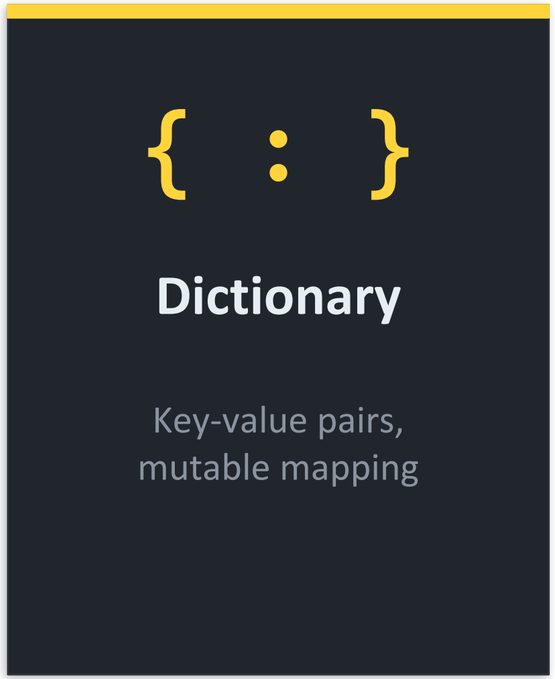
07 **Stack vs Queue**

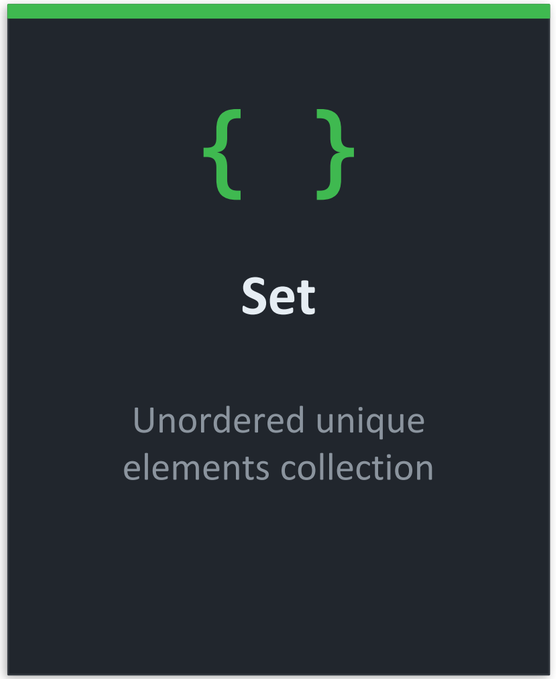
Comparison & when to use each

Python's Core Data Structures









Property	List	Tuple	Dict	Set
Ordered	✓	✓	✓ (3.7+)	✗
Mutable	✓	✗	✓	✓
Duplicates	✓	✓	Keys: ✗	✗
Indexed	✓	✓	By key	✗

List — Ordered & Mutable Sequence

What is a List?

1. Stores an ordered collection of items
2. Elements can be of any data type
3. Mutable: add, remove, or change elements
4. Supports duplicate values
5. Uses zero-based integer indexing
6. Supports negative indexing (-1 = last)
7. Dynamic: grows or shrinks automatically

```
# Creating lists
fruits = ["apple", "banana", "cherry"]
mixed  = [1, "hello", 3.14, True]
nested = [[1, 2], [3, 4], [5, 6]]
empty  = []

# Indexing
print(fruits[0])     # apple
print(fruits[-1])    # cherry

# Slicing  [start:stop:step]
print(fruits[0:2])   # ['apple', 'banana']
print(fruits[::-1])  # reverse the list
```

💡 Memory Tip

Lists are stored as dynamic arrays in CPython. Appending is O(1) amortized, but insertion at position 0 is O(n).

Stack

Last In, First Out (LIFO)

📌 Key Properties

- Insert (push) and remove (pop) from the TOP only
- Last element inserted is the FIRST to be removed
- peek() views the top without removing
- isEmpty() checks if stack is empty

🥞 Real-World Analogy

Stack of plates
You always take the TOP plate. You always put the new plate on TOP.

Call stack in programs
Functions are pushed when called, popped when they return.

Stack in Python

3 Implementation Methods

① list (simplest)

```
stack = []
stack.append(10)   # push
stack.append(20)
stack.append(30)
top = stack[-1]    # peek → 30
val = stack.pop()  # pop  → 30
print(stack)       # [10, 20]
```

② collections.deque (recommended)

```
from collections import deque
stack = deque()
stack.append(10)   # push
stack.append(20)
top = stack[-1]    # peek → 20
val = stack.pop()  # pop  → 20
print(stack)       # deque([10])
```

③ queue.LifoQueue (thread-safe)

```
from queue import LifoQueue
stack = LifoQueue()
stack.put(10)      # push
stack.put(20)
val = stack.get()  # pop  → 20
print(stack.empty())  # False
```

💡 Recommendation: Use collections.deque for O(1) append/pop performance. Python list is fine for small stacks but deque is more efficient for large data.

stack_demo.py

```python
from collections import deque


class Stack:

    def __init__(self):

        self._data = deque()


    def push(self, val):

        self._data.append(val)


    def pop(self):

        if self.is_empty(): raise IndexError

        return self._data.pop()
```

Output

```
30
30
```

push(x)
O(1)
Add x to top

pop()
O(1)
Remove top

peek()
O(1)
View top

is_empty()
O(1)
Check empty

Stack Applications

↩ Undo / Redo

Text editors push each action onto a stack. Undo pops the last action.

```
history.append(action)  # do
action = history.pop()   # undo
```

( ) Bracket Matching

Parse '(', '[', '{' — push opens, pop on close and verify they match.

```
for ch in expr:
  if ch in '([{': stk.append(ch)
  else: stk.pop()
```

📞 Function Call Stack

The OS pushes each function call; pops on return. Drives recursion.

```
# Python sys module
import sys
print(sys.getrecursionlimit())
```

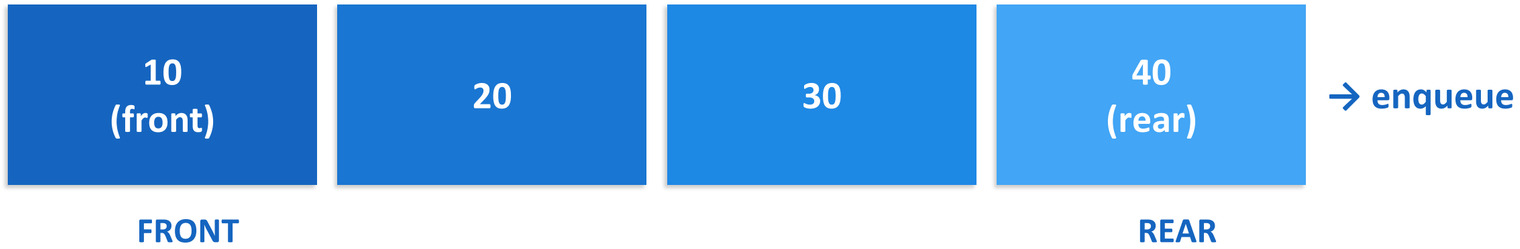
🔍 DFS (Graph Traversal)

Depth-First Search uses a stack to track nodes to visit next.

```
stack = [start]
while stack:
  node = stack.pop()
  visit(node)
```

Queue

First In, First Out (FIFO)



📌 Key Properties

- Insert (enqueue) at the REAR
- Remove (dequeue) from the FRONT
- First element inserted is FIRST to be removed
- peek() views the front without removing

Real-World Analogy

Ticket queue at a cinema
First person in line is first to be served.

CPU task scheduling
OS queues processes in the order they arrive.

Queue in Python

3 Implementation Methods

① list (avoid for large data)

```
queue = []
queue.append(10)    # enqueue rear
queue.append(20)
queue.append(30)
val = queue.pop(0)  # dequeue front
print(val)          # 10
# ⚠ pop(0) is O(n) — slow!
```

② collections.deque (recommended)

```
from collections import deque
q = deque()
q.append(10)       # enqueue rear
q.append(20)
val = q.popleft()  # dequeue front
print(val)         # 10
# ✅ Both ends O(1)
```

③ queue.Queue (thread-safe)

```
from queue import Queue
q = Queue()
q.put(10)          # enqueue
q.put(20)
val = q.get()      # dequeue → 10
print(q.qsize())   # 1
print(q.empty())   # False
```

💡 Recommendation: Use collections.deque — popleft() is O(1) unlike list's pop(0) which is O(n). Use queue.Queue for multi-threaded applications.

queue_demo.py

```python
from collections import deque


class Queue:

    def __init__(self):

        self._data = deque()


    def enqueue(self, val):

        self._data.append(val)


    def dequeue(self):

        if self.is_empty(): raise IndexError

        return self._data.popleft()
```

Output

```
10
10
```

enqueue(x)
O(1)
Add to rear

dequeue()
O(1)
Remove front

peek()
O(1)
View front

is_empty()
O(1)
Check empty

Queue Applications

BFS (Graph Traversal)

Breadth-First Search explores level by level using a queue for nodes.

```
q = deque([start])
while q:
  node = q.popleft()
  q.extend(neighbors(node))
```

Task / Job Scheduling

OS process schedulers queue tasks in arrival order (Round Robin, FCFS).

```
job_queue = deque()
job_queue.append('job_A')
job_queue.append('job_B')
cpu.run(job_queue.popleft())
```

Network Packet Buffer

Routers and switches queue packets; process in order they arrive.

```
buffer = deque(maxlen=1024)
buffer.append(packet)
process(buffer.popleft())
```

Print Spooler

Print jobs are queued. The first document sent is printed first.

```
print_q = Queue()
print_q.put('doc1.pdf')
print_q.put('doc2.pdf')
printer.run(print_q.get())
```

Stack vs Queue

STACK		QUEUE
LIFO — Last In, First Out	Principle	FIFO — First In, First Out
push() — adds to TOP	Insert	enqueue() — adds to REAR
pop() — removes from TOP	Remove	dequeue() — removes from FRONT
One end only (top)	Access	Two ends (front & rear)
list / deque / LifoQueue	Python impl	deque / list / Queue
Undo, DFS, recursion	Use cases	BFS, scheduling, buffers
Stack of plates	Analogy	Queue at a cinema

Summary

Stack (LIFO)

- push() / pop() / peek() — all O(1)
- Use deque for best performance
- Applications: undo, DFS, call stack, bracket matching

Queue (FIFO)

- enqueue() / dequeue() / peek() — all O(1)
- Use deque.popleft() — never list.pop(0)
- Applications: BFS, task scheduling, buffers

collections.deque is your best friend for both Stack and Queue in Python!

Part-2

Python Data Structures

LeetCode Problem "Valid Parentheses"

Course Activities
Github repository
Discussion Forum

LeetCode Practice

Master Programming Skills with Python · C++ · Rust

2500+
Problems

50+
Companies

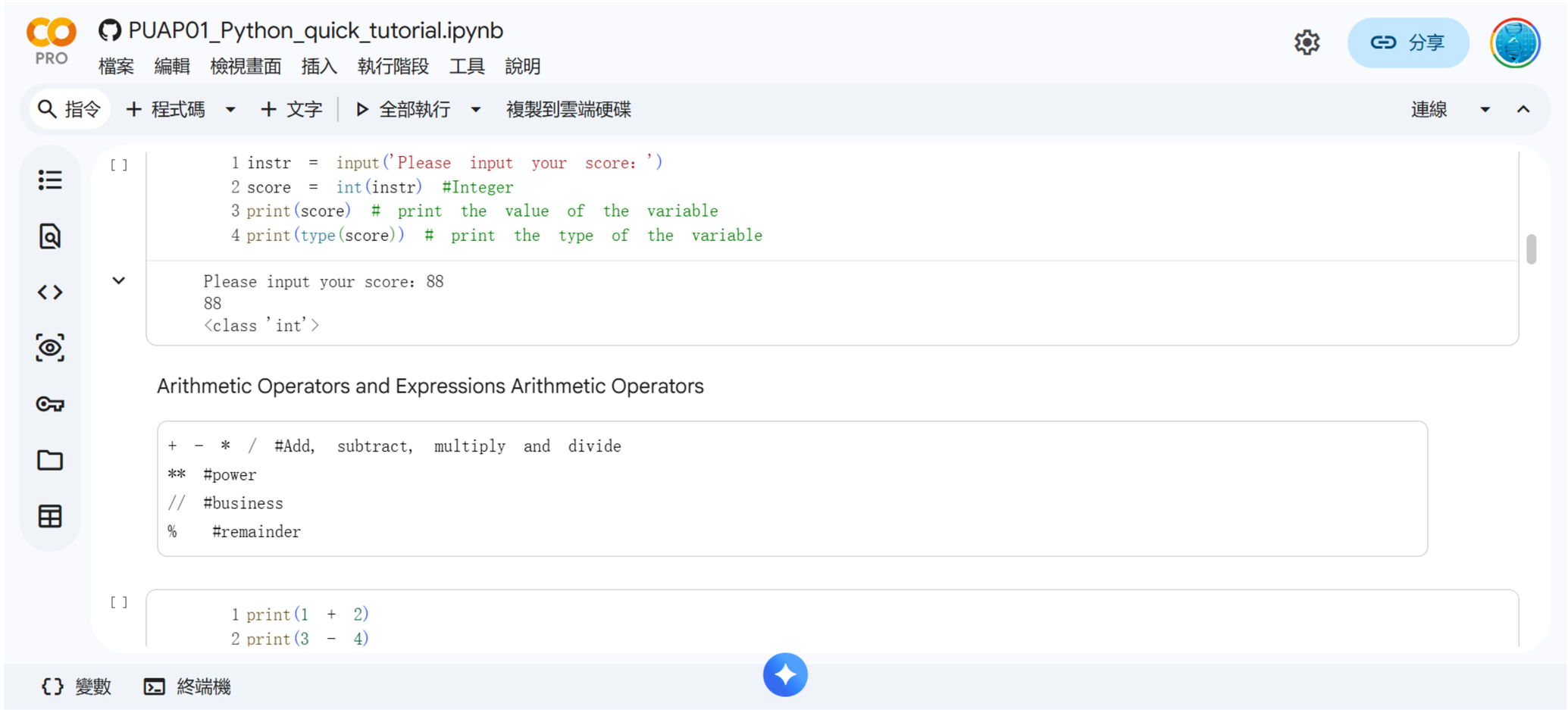
10M+
Users

Difficulty Tiers

- Easy
- Medium
- Hard

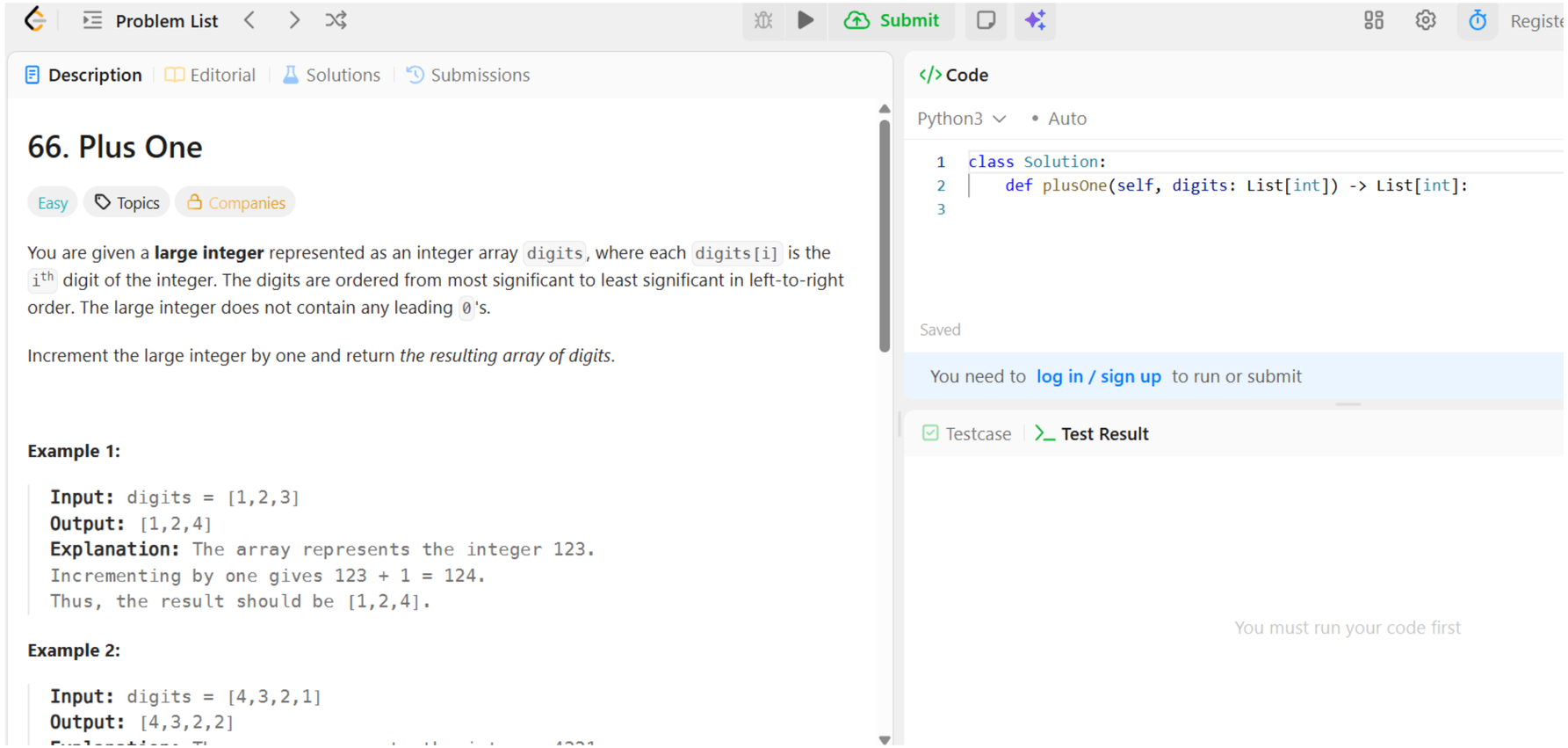
Arrays · Strings · Trees · Graphs · DP · Greedy · Divide & Conquer · Two Pointers · Sliding Window

Two Sum



https://leetcode.com/problems/two-sum/description/
https://leetcode.cn/problems/two-sum/description/

Plus One



https://leetcode.com/problems/plus-one/description/
https://leetcode.cn/problems/plus-one/description/

Valid Parentheses

https://leetcode.com/problems/valid-parentheses/
https://leetcode.cn/problems/valid-parentheses/

#20 · LeetCode | Easy

Valid

Parentheses

Topic	Difficulty	Time	Space
Stack	Easy	O(n)	O(n)

Given a string of brackets, determine if it is valid.

() ✓

()[]{} ✓

(] ✗

([)] ✗

{[]} ✓

Problem Statement

EASY

Given a string s containing just the characters `'('`, `')'`, `'{'`, `'}'`, `'['` and `']'`

determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of bracket.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket.

Constraints: $1 \le s.length \le 10^4$ · s consists of parentheses only `'()[]{}'`

Examples

I/O

Example 1 ✓ **TRUE**

```
Input:  "()"
Output: true
```

Single matching pair — (closed by).

Example 2 ✓ **TRUE**

```
Input:  "()[]{}"
Output: true
```

Three separate pairs, each closed correctly.

Example 3 ✗ **FALSE**

```
Input:  "(]"
Output: false
```

(expects), but gets] — wrong type.

Example 4 ✗ **FALSE**

```
Input:  "([)]"
Output: false
```

Brackets interleave — inner pair not closed before outer.

Example 5 ✓ **TRUE**

```
Input:  "{[]}"
Output: true
```

Nested brackets closed in reverse order — correct.

Key Insight — Why a Stack?

LIFO

The most recently opened bracket must be closed first. This is exactly what a **Stack (LIFO)** gives us.

Last In, First Out = The last bracket pushed is the first one we need to match.

Open Bracket?

→ PUSH

1. Encounter ([or {
2. Push it onto the stack
3. We will match it later

Close Bracket?

← POP

1. Encounter)] or }
2. Pop the top of the stack
3. Must match the close bracket

End of String?

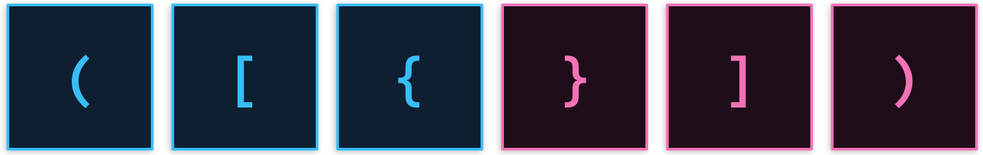
✓ CHECK

1. All chars processed
2. Stack must be empty
3. Any leftover = INVALID

Matching pairs: (→) [→] { → }

Walkthrough — "([{}])" → true

VALID ✓



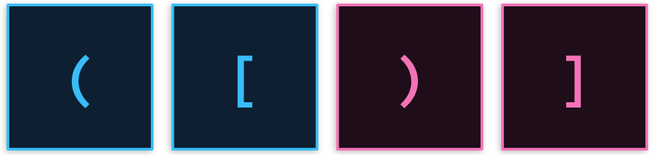
Input string:

Step	Char	Action	Stack after
1	(	PUSH → stack not empty	(↑top
2	[	PUSH → stack grows	([↑top
3	{	PUSH → 3 open brackets	([{ ↑top
4	}	POP → top { matches } ✓	([↑top
5	]	POP → top [matches] ✓	(↑top
6	)	POP → top (matches) ✓	[empty]

✓ Stack is empty after processing all characters → return True

Walkthrough — "([)]" → false

INVALID ✗



Input string:

Step	Char	Action	Stack
1	(	PUSH → open bracket	(
2	[	PUSH → open bracket	([
3	)	POP → top is [, expects] ✗ MISMATCH!	STOP ✗
4	]	Return false immediately	

Why is this invalid?

When we see) at index 2, the TOP of the stack is [(from index 1).
) expects (on top, but we found [— this is a TYPE mismatch.

✗ Mismatch detected → return False immediately (no need to continue)

Edge Cases to Handle

TRICKY

✗ Empty Stack on Close

Input: `")"` Output: **false**

First char is) but stack is empty — no matching open bracket exists.
Always check stack is not empty before popping.

✗ Leftover Open Brackets

Input: `"((("` Output: **false**

Three open brackets pushed, never closed.
After loop ends, stack is non-empty → return False.

✗ Single Character

Input: `"("` Output: **false**

One open bracket with no close — stack has 1 item at end → False.
Any odd-length string is automatically False.

✓ Properly Nested

Input: `"{[]}"` Output: **true**

{ pushed, [pushed,] pops [✓, } pops { ✓ — stack empty → True.
Deep nesting is fine as long as order is correct.

Golden rule: at the end, stack must be EMPTY to return True — otherwise False.

Algorithm (Pseudocode)

LOGIC

pseudocode

```
function isValid(s):
  stack ← empty stack
  pairs ← { ')':'(', ']':'[', '}':'{' }

  for each char c in s:
    if c is an OPEN bracket  (  [  {
      push c onto stack
    else  (c is a CLOSE bracket  )  ]  })
      if stack is empty:
        return False  ← no matching open
      if stack.top ≠ pairs[c]:
        return False  ← wrong type
      pop from stack

  return stack is empty
```

Open bracket?

↓ **PUSH to stack**

Close bracket + empty stack?

✗ **Return FALSE**

Close bracket + wrong top?

✗ **Return FALSE**

Close bracket + correct top?

✓ **POP from stack**

End of string + empty?

✓ **Return TRUE**

End of string + non-empty?

✗ **Return FALSE**

Python Solution

CODE

solution.py

```python
class Solution:
    def isValid(self, s: str) -> bool:
        stack = []
        pairs = {')': '(', ']': '[', '}': '{'}

        for char in s:
            if char in pairs.values():
                stack.append(char)
            elif char in pairs:
                if not stack or stack[-1] != pairs[char]:
                    return False
                stack.pop()

        return not stack
```

Stack

Python list used as a stack — append() = push, pop() = pop

Pair Map

Dictionary maps each CLOSE bracket to its expected OPEN bracket

Loop

Iterate over every character in the string one by one

Open bracket

push to stack so we can match it later

Close bracket

Check: stack non-empty AND top equals the expected pair

Final check

'not stack' is True only if stack is empty — all matched!

Complexity Analysis

BIG-O

⏱ Time Complexity

O(n)

where n = length of string s

- We visit each character exactly once
- Each char triggers one push or one pop
- Stack push/pop are both O(1)
- Total: n iterations × O(1) = O(n)

💾 Space Complexity

O(n)

worst case: all open brackets

- Stack stores open brackets seen so far
- Worst: "(((((" — all n chars on stack
- Best: "()" — stack never exceeds 1
- Auxiliary space for hash map: O(1) fixed

Both Time and Space are O(n) — optimal for this problem since we must read every character at least once.

Common Mistakes

BUGS

✗ Forgetting to check empty stack on close

✗ wrong

```
if stack[-1] != pairs[c]:  # IndexError!
    return False
```

✓ correct

```
if not stack or stack[-1] != pairs[c]:
    return False  # safe!
```

✗ Returning True inside the loop

✗ wrong

```
for c in s:
    if c == ')' and stack and stack[-1]=='(':
        stack.pop()
        return True  # too early!
```

✓ correct

```
for c in s:
    # ... process ...
return not stack  # after the loop
```

✗ Returning True when stack is non-empty

✗ wrong

```
for c in s:
    # ... process ...
return True  # forgot to check!
```

✓ correct

```
for c in s:
    # ... process ...
return len(stack) == 0  # correct
```

Summary

#20 · Valid Parentheses

📋 Problem

Check if a bracket string is valid — all brackets must be correctly matched and properly nested.

💡 Insight

Most recently opened bracket must close first — exactly the Last In First Out (LIFO) behaviour of a Stack.

⚙️ Algorithm

Push open brackets. On close bracket: check stack top matches, then pop. Return stack.empty() at end.

🐍 Python

Use a list as a stack. Store closing→opening map. Use stack[-1] for peek. "not stack" to check empty.

⏱ Time O(n)

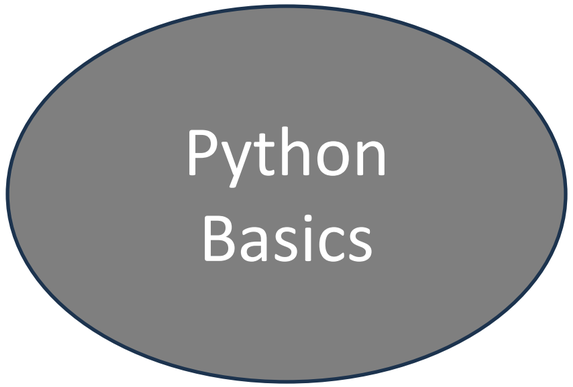
One pass through the string. Each character is pushed or popped exactly once — all O(1) operations.

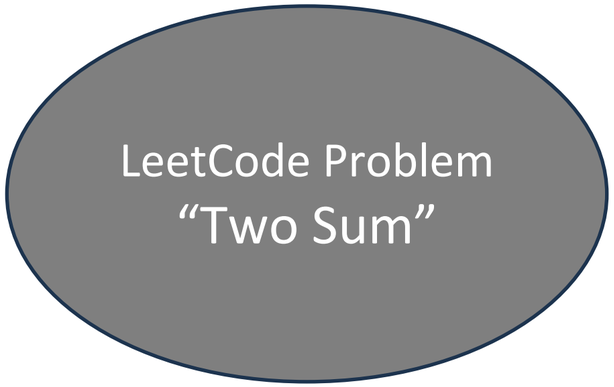
💾 Space O(n)

Stack holds at most n/2 brackets (worst case all openers). Hash map is O(1) fixed size.

Tip: Valid Parentheses is the classic introduction to Stack problems — master it and you'll solve dozens of harder variants!

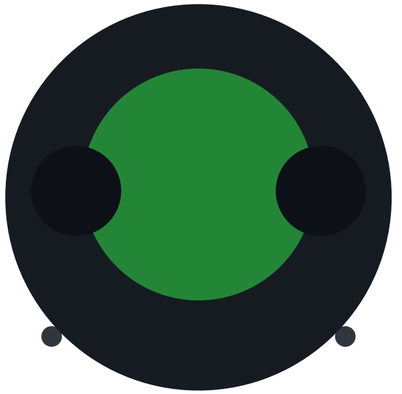
Part-3


Python
Basics


LeetCode Problem
“Two Sum”


Course Activities
Github repository
Discussion Forum

GitHub Desktop



A Visual Guide to Git Workflow

No command line required · Beginner friendly · Cross-platform

Clone | Commit | Branch | Push / Pull | Pull Request

What is GitHub Desktop?

Definition

GitHub Desktop is a free, open-source GUI application that lets you interact with GitHub and Git repositories without using the command line.

- Visual diff — see exactly what changed
- One-click commit, push, and pull
- Branch management made simple
- Built-in merge conflict editor
- Seamless GitHub integration

Visual

See file diffs in colour, not plain text

Mouse-driven

No terminal commands to memorise

Branching

Create & switch branches in one click

GitHub sync

Integrated login, PRs & notifications

Installation & Setup

Step-by-step

1 Download

Go to desktop.github.com and click Download for macOS or Windows.

✓ Free · No account needed to download

2 Install

Run the installer. On Mac, drag to Applications. On Windows, follow the setup wizard.

✓ Installs Git automatically if missing

3 Sign In

Open GitHub Desktop → File → Options → Accounts → Sign in to GitHub.com.

✓ Supports GitHub.com & GitHub Enterprise

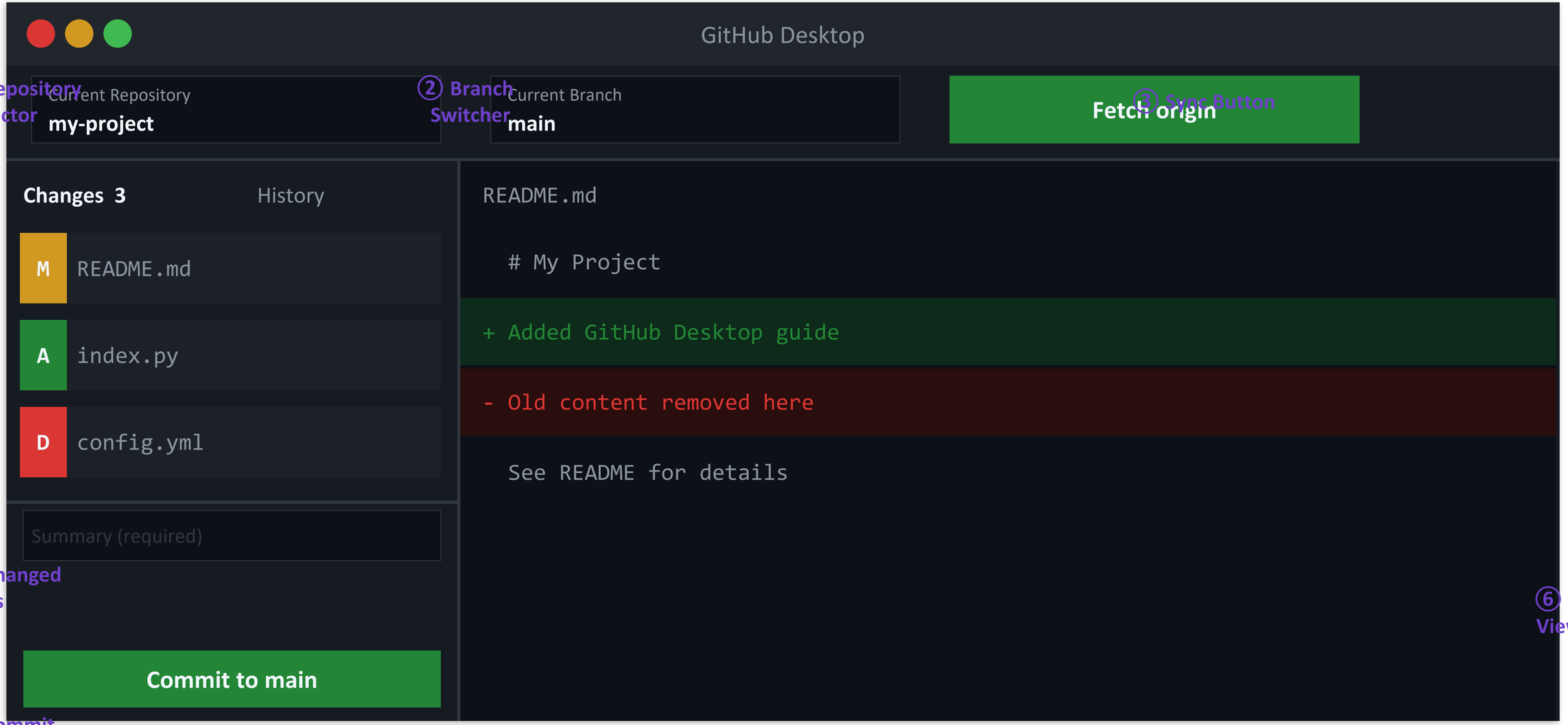
4 Configure Git

Set your Name and Email in Git tab under Options. These appear on all your commits.

✓ Matches your GitHub profile

💡 Tip: GitHub Desktop ships with its own bundled Git — you don't need to install Git separately on Windows.

Interface Overview



Cloning a Repository

Getting code onto your machine

Method A — From GitHub.com

1. Navigate to any repository on GitHub.com
2. Click the green Code button
3. Select Open with GitHub Desktop
4. Choose a local folder path
5. Click Clone — done!

Method B — Inside GitHub Desktop

1. Go to File → Clone Repository…
2. Search by name or paste a URL
3. Browse and select a local folder
4. Click Clone
5. Repository is ready to use

📁 Clone destination default: macOS → ~/Documents/GitHub/ | Windows → C:\Users\<you>\Documents\GitHub\

Making Changes & Committing

Edit files in your editor → GitHub Desktop detects changes → Stage files (tick boxes) → Write commit message → Click Commit to branch

✅ Writing Good Commit Messages

✓ `Fix login bug on mobile`

✓ `Add dark mode toggle to settings`

✗ `Fix stuff`

✗ `asdfasdf`

✗ `Updated files`

Use imperative tense · Under 72 chars · Describe the WHY

📸 What is a Commit?

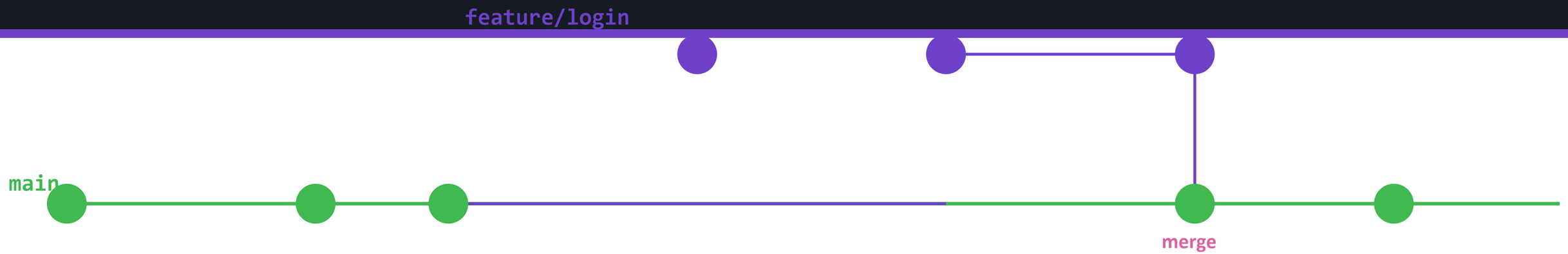
A commit is a snapshot of your project at a point in time.
Each commit stores:

- What changed (the diff)
- Who made the change (author)
- When it happened (timestamp)
- A unique ID (SHA hash)
- A link to the previous commit

```
a3f7c2b  Fix login bug  (you, 2 min ago)
```

Working with Branches

Isolate work · Experiment safely



Create a Branch

1. Click Current Branch in toolbar
2. Click New Branch
3. Name it: feature/your-feature
4. Click Create Branch

Switch Branches

1. Click Current Branch
2. Select the branch you want
3. GitHub Desktop updates your files instantly
4. Uncommitted changes may carry over

Delete a Branch

1. Make sure it's merged first
2. Right-click the branch name
3. Choose Delete…
4. Optionally delete on remote too

💡 Best practice: Never commit directly to main. Always work in a branch and merge via a Pull Request.

Push & Pull

💻 Local

Your computer
(GitHub Desktop)

```
~/Documents/GitHub/project
```

PUSH →

Send local commits

← PULL

Fetch + merge remote

☁ Remote

GitHub.com server
(shared with team)

```
github.com/user/project
```

Push Origin

After committing, click Push origin in the toolbar to upload your commits to GitHub.com.

💡 The toolbar shows "↑ 2" if you have 2 unpushed commits.

Pull / Fetch

"Fetch" checks for remote changes. "Pull" downloads them and merges into your current branch.

💡 Do this before starting work each day to stay in sync.

Force Push

Overwrites remote history. ONLY use on your own personal branches — NEVER on main.

💡 Repository → Push — hold Alt/Option to reveal force push.

Resolving Merge Conflicts

When two branches edit the same line

⚠ When do conflicts happen?

Two branches modify the same line of the same file, and Git cannot automatically decide which version to keep.

conflict_example.py

```
<<<<<<< HEAD
print('Hello from main')
=======
print('Hello from feature')
>>>>>>> feature/login
```

1 GitHub Desktop Alerts You

After a merge attempt, conflicted files are listed with a warning badge ⚠

2 Open in Editor

Click Open in Editor to see conflict markers inside the file

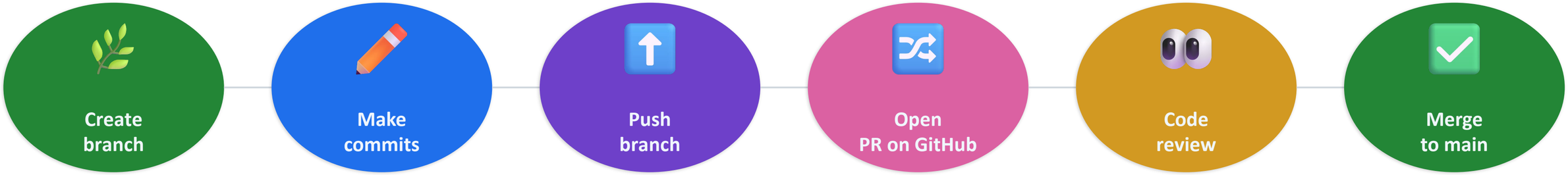
3 Edit & Resolve

Remove conflict markers, keep the correct code, save the file

4 Mark as Resolved

Return to GitHub Desktop, click Mark as resolved, then commit the merge

Pull Requests (PRs)



Creating a PR from GitHub Desktop

1. Push your branch to GitHub
2. GitHub Desktop shows "Create Pull Request" banner
3. Click it — GitHub.com opens in your browser
4. Fill in title and description
5. Assign reviewers (optional)
6. Click Create pull request

Anatomy of a Good PR

Title: Short, imperative: Add dark-mode toggle

Description: What changed, why, and how to test it

Screenshots: For UI changes — show before & after

Reviewers: Tag team members who should review

Labels: bug, enhancement, documentation…

Tips & Best Practices

Work smarter with GitHub Desktop

🔄 Sync Daily

Click Fetch origin every morning before you start. Avoids large conflicts later.

🌿 One Branch = One Feature

Keep branches small and focused. Merge quickly to avoid drift from main.

📝 Commit Often

Small frequent commits are easier to review, revert, and understand than large ones.

🔍 Review Diffs Before Commit

Always check the diff panel — catch accidental debug code or secrets before pushing.

🏷️ Use Descriptive Branch Names

Names like feature/user-login or bugfix/null-pointer make history readable.

🚫 Never Force-Push to main

Force-pushing rewrites history and can destroy teammates' work permanently.

Summary

Clone

Get a remote repo onto your machine with one click

Branch

Isolate every feature or fix in its own branch

Commit

Snapshot changes with a clear, descriptive message

Push

Upload your commits to GitHub.com for backup & sharing

Pull

Download teammates' changes and stay up to date

PR & Merge

Request review, merge into main, keep history clean

GitHub Desktop makes Git accessible to everyone — start with Clone, commit often, and collaborate through Pull Requests.